

Neuroalianza — Arquitectura de Backend

Hackatón Niño San Borja 2026 — Desafío 04: Neurodesarrollo

Documento de especificación técnica · Versión 1.0

1. Propósito del documento

Este documento define la arquitectura del backend de Neuroalianza: una plataforma que conecta al personal de salud del primer nivel, a las familias y al equipo multidisciplinario especializado alrededor de un mismo caso de neurodesarrollo.

El documento especifica **qué debe hacer el backend, cómo se organiza internamente y bajo qué reglas de calidad se construye**. No es un manual de implementación: es el contrato que permite que varias personas trabajen en paralelo sin colisionar y que el sistema pueda evolucionar de un prototipo de hackatón a un sistema productivo sin reescribirse.

1.1 Restricciones de partida

Restricción	Implicación arquitectónica
El sistema no diagnostica	El motor de tamizaje produce niveles de riesgo y recomendaciones de acción, nunca etiquetas clínicas
No se integra con sistemas reales durante el evento	Todo servicio externo se consume detrás de un contrato, con implementación simulada por defecto
Conectividad limitada en zonas alejadas	La API debe tolerar sincronización diferida y operaciones idempotentes
Sin infraestructura costosa	El sistema completo debe arrancar con un único proceso y cero dependencias externas
Debe demostrarse en vivo	El paso del tiempo debe poder simularse para disparar alertas frente al jurado

2. Principios arquitectónicos

2.1 Puertos y adaptadores

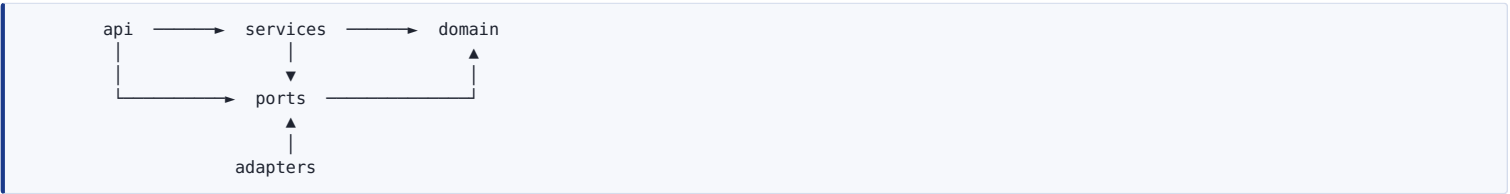
El núcleo del sistema —el dominio— no conoce ninguna tecnología externa. Declara **puertos**: interfaces abstractas que describen lo que necesita ("persistir un caso", "enviar un aviso"). Los **adaptadores** las implementan.

Este principio no es decorativo. Sostiene tres propiedades que el proyecto necesita:

- Arranque inmediato.** Los adaptadores en memoria permiten ejecutar el sistema completo sin base de datos, sin credenciales y sin red.
- Sustitución sin refactor.** Migrar a producción es implementar el mismo contrato con otra tecnología y cambiar una variable de entorno.
- Testabilidad.** El dominio se prueba sin levantar infraestructura, con tests que corren en milisegundos.

2.2 Regla de dependencia

Las dependencias apuntan **siempre hacia adentro**. Ninguna capa interior importa una exterior.



Capa	Puede importar	Nunca importa
domain	nada del proyecto (solo stdlib)	ports , adapters , services , api
ports	domain	adapters , services , api
services	domain , ports	adapters , api
adapters	domain , ports	services , api
api	domain , ports , services	adapters (salvo en el arranque)

Esta regla se verifica automáticamente en CI mediante `import-linter` (sección 11.3). No es una convención de buena voluntad: es una prueba que falla el build.

2.3 Modularidad por contexto, no por tipo técnico

Dentro del dominio, la separación no es por artefacto (`models/`, `services/`, `utils/`) sino por **contexto funcional**: tamizaje, caso, agenda, notificaciones, métricas. Cada contexto es un módulo con su propia frontera y su propio lenguaje.

Un módulo bien delimitado cumple tres pruebas:

- Se puede explicar en una frase sin usar la conjunción "y"
- Sus tests no requieren construir objetos de otro módulo
- Eliminarlo rompe una funcionalidad completa, no medias funcionalidades de varias

2.4 Comunicación entre módulos por eventos

Los módulos no se llaman entre sí directamente. Publican eventos de dominio en un bus, y quien esté interesado se suscribe.

Cuando un caso pasa a `DERIVADO`, el módulo de casos no sabe que existen notificaciones ni métricas. Publica `CasoDerivado` y sigue. Los módulos de notificación y métricas reaccionan. Agregar un cuarto comportamiento no toca el flujo principal.

3. Estructura de módulos

```
app/
├── main.py                # composición y arranque de FastAPI
├── config.py              # settings tipadas, selección de adaptadores
├── container.py           # fábrica de dependencias (composition root)
├──
├── domain/               # núcleo puro – cero imports externos
│   ├── shared/
│   │   ├── ids.py        # tipos de identidad (CasoId, PacienteId, ...)
│   │   ├── errors.py     # excepciones de dominio
│   │   └── result.py     # tipo Result para operaciones fallibles
│   ├── screening/
│   │   ├── catalog.py    # catálogo de ítems por rango etario
│   │   ├── rules.py      # reglas de puntaje → nivel de riesgo
│   │   └── models.py     # Cuestionario, Respuesta, ResultadoTamizaje
│   ├── casefile/
│   │   ├── models.py     # Caso, Paciente, Derivacion, NotaClinica
│   │   ├── states.py     # máquina de estados
│   │   └── timeline.py   # historial inmutable de transiciones
│   ├── scheduling/
│   │   ├── models.py     # Cita, Disponibilidad, BloqueDeSesiones
│   │   └── rules.py      # agrupación de sesiones, detección de choques
│   ├── alerts/
│   │   └── rules.py      # reglas temporales de alerta
│   └── events.py         # catálogo de eventos de dominio
├──
├── ports/                # contratos abstractos
│   ├── repositories.py
│   ├── notifier.py
│   ├── event_bus.py
│   ├── clock.py
│   ├── file_storage.py
│   └── unit_of_work.py
├──
├── adapters/
│   ├── memory/           # implementación por defecto
│   ├── postgres/        # producción
│   ├── messaging/       # WhatsApp / SMS / correo
│   └── storage/          # S3 u object storage
├──
├── services/             # casos de uso – orquestan dominio + puertos
│   ├── screening_service.py
│   ├── referral_service.py
│   ├── scheduling_service.py
│   ├── casefile_service.py
│   ├── notification_service.py
│   ├── alert_engine.py
│   └── metrics_service.py
├──
├── api/
│   ├── deps.py           # inyección de dependencias
│   ├── errors.py         # manejadores de error → RFC 7807
│   ├── schemas/          # modelos Pydantic de request/response
│   └── routes/
│       ├── health_worker.py # personal de salud del primer nivel
│       ├── family.py       # familia
│       ├── specialist.py   # equipo multidisciplinario
│       ├── admin.py        # operaciones de demo y administración
│       └── system.py       # salud, versión, diagnóstico
├──
└── seed/
    └── demo_data.py       # dataset precargado para la demostración
```

4. Modelo de dominio

4.1 Entidades principales

Paciente. Identidad del niño o adolescente. Datos mínimos: identificador, fecha de nacimiento, sexo, ubicación (departamento, provincia, distrito), establecimiento de origen. Sin datos sensibles innecesarios: el principio es minimización.

Cuidador. Persona responsable. Canal de contacto preferido y su disponibilidad horaria. Un paciente puede tener varios; uno es el contacto primario.

Caso. La unidad central del sistema. Agrupa a un paciente con todo su recorrido: tamizajes, derivación, citas, evaluaciones, diagnóstico, terapias y seguimiento. Un paciente puede tener más de un caso a lo largo del tiempo, pero solo uno activo.

Transición. Registro inmutable de un cambio de estado del caso. Contiene: estado origen, estado destino, actor que la produjo, marca de tiempo, motivo y metadatos. **Nunca se modifica ni se elimina.** El historial completo de transiciones es la trazabilidad, y de él se derivan todas las métricas.

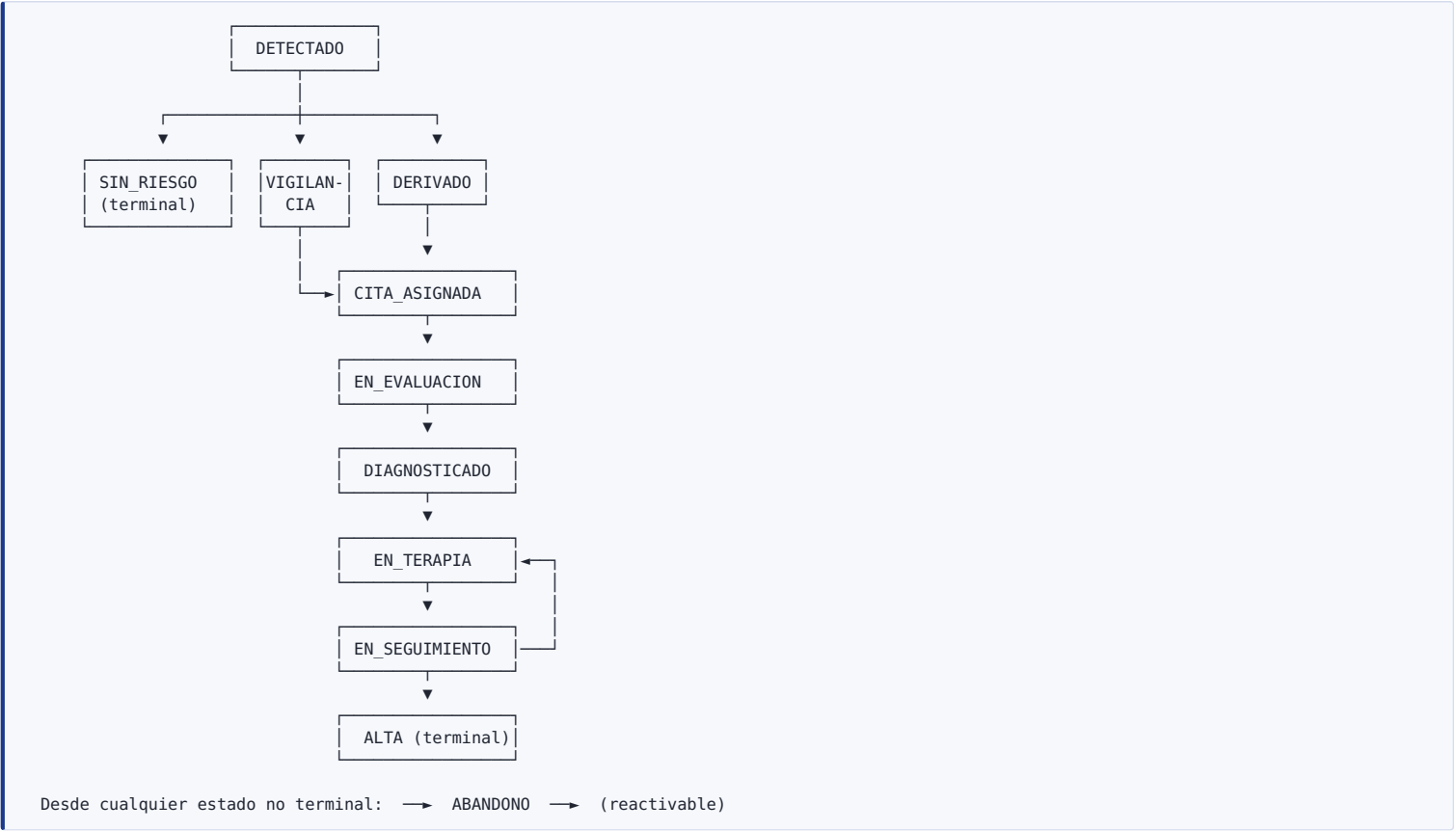
ResultadoTamizaje. Producto de aplicar el cuestionario. Contiene el nivel de riesgo, los ítems que se activaron, la acción recomendada y la versión del catálogo utilizada. Esa versión es indispensable: permite reinterpretar resultados históricos cuando las reglas cambien.

Cita. Encuentro programado. Tipo (evaluación, terapia, control), profesional, establecimiento, modalidad (presencial o remota), estado (programada, confirmada, atendida, no asistida, reprogramada) y —si aplica— motivo de inasistencia.

NotaClinica. Registro de un profesional sobre el caso. Tiene autor, especialidad, contenido y nivel de visibilidad.

PlanTerapeutico. Conjunto de indicaciones y actividades de refuerzo en el hogar, con periodicidad y registro de cumplimiento.

4.2 Máquina de estados del caso



Reglas de la máquina:

- Las transiciones válidas se declaran explícitamente. Cualquier otra es rechazada con un error de dominio.
- Cada transición exige un actor identificado y un motivo.
- ABANDONO es alcanzable desde cualquier estado activo y es reactivable: un caso reactivado retoma el estado previo, conservando el registro del abandono.
- La máquina es una función pura sobre el estado actual y el evento solicitado. No accede a persistencia ni a red.

Invariantes:

- Un caso siempre tiene exactamente un estado actual
- El historial de transiciones es append-only
- La marca de tiempo de una transición nunca es anterior a la de la transición previa
- Un paciente no puede tener dos casos activos simultáneamente

4.3 Motor de tamizaje

Es la pieza que más escrutinio recibirá de un jurado clínico, y por eso su diseño prioriza **auditabilidad** sobre sofisticación.

Entrada: edad del paciente en meses, conjunto de respuestas, versión del catálogo.

Proceso: 1. Se selecciona el conjunto de ítems correspondiente al rango etario 2. Se valida que todos los ítems obligatorios tengan respuesta 3. Se aplican las reglas de puntaje declaradas en el catálogo 4. Se determina el nivel de riesgo

Salida: nivel de riesgo (BAJO , MODERADO , ALTO), lista de ítems activados con su justificación, acción recomendada, y versión del catálogo aplicada.

Requisitos no negociables:

- El motor es una **función pura**: mismas entradas, misma salida, sin efectos.
- Las reglas viven en datos versionados, no incrustadas en el código.
- La salida **nunca** contiene una etiqueta diagnóstica. Se expresa en términos de acción: *observar y reevaluar en N semanas, derivar a evaluación especializada, sin señales de alarma en esta aplicación*.
- Todo resultado es explicable: para cada nivel de riesgo, el sistema puede enumerar qué ítems lo produjeron.

El catálogo debe construirse sobre instrumentos ya reconocidos en el contexto peruano (EEDP, TEPSI, M-CHAT-R/F, pauta breve de MINSA). Usar instrumentos validados en lugar de una escala propia es una decisión arquitectónica y también una decisión de credibilidad.

4.4 Reglas de alerta

Las alertas son reglas temporales evaluadas periódicamente contra el estado de los casos. Cada una es un objeto con: identificador, condición,

severidad, destinatarios y política de repetición.

Regla	Condición	Destinatario
Derivación estancada	DERIVADO por más de N días sin cita asignada	Coordinación del centro especializado
Recordatorio de cita	Cita programada dentro de las próximas 24/48 h	Cuidador primario
Inasistencia reiterada	Dos o más citas no asistidas consecutivas	Equipo tratante
Riesgo de abandono	Sin actividad registrada en N semanas estando en terapia	Equipo tratante + establecimiento de origen
Reevaluación vencida	En VIGILANCIA con fecha de reevaluación superada	Personal de salud que detectó
Evaluación incompleta	Bloque de sesiones iniciado sin completar en N semanas	Coordinación
Contrarreferencia pendiente	Caso diagnosticado sin contrarreferencia enviada	Especialista responsable

Los umbrales son configuración, no código.

4.5 Catálogo de eventos de dominio

Evento	Se publica cuando	Consumidores
TamizajeAplicado	Se registra un cuestionario	Métricas, Caso
CasoCreado	Nace un caso	Métricas
CasoDerivado	Transición a DERIVADO	Notificaciones, Métricas, Alertas
CitaAsignada	Se programa una cita	Notificaciones, Métricas
CitaConfirmada	El cuidador confirma	Métricas
CitaNoAsistida	Se marca inasistencia	Alertas, Métricas
InasistenciaJustificada	El cuidador reporta motivo	Métricas, Alertas
EvaluacionIniciada / Completada	Bloque de evaluación	Métricas
DiagnosticoRegistrado	Se registra conclusión diagnóstica	Notificaciones, Métricas
PlanTerapeuticoDefinido	Se establece el plan	Notificaciones
ContrarreferenciaEnviada	Se comunica al origen	Notificaciones, Métricas
CasoAbandonado / Reactivado	Cambio a/desde abandono	Alertas, Métricas

Todo evento lleva: identificador único, tipo, marca de tiempo, identificador del caso, actor y carga útil tipada.

5. Puertos

Los puertos son clases abstractas (Protocol o ABC) definidas en términos del dominio. Ninguna firma menciona SQL, HTTP, colas ni formatos de terceros.

5.1 Repositorios

Un repositorio por agregado. Firma mínima:

Operación	Descripción
add(entidad)	Registra una entidad nueva
get(id)	Recupera por identidad; error de dominio si no existe
find(criterio)	Búsqueda por especificación tipada
update(entidad)	Persiste cambios

Repositorios requeridos: PacienteRepository , CasoRepository , CitaRepository , TamizajeRepository , NotificacionRepository , UsuarioRepository .

Los criterios de búsqueda se expresan como objetos de especificación, no como cadenas de consulta. Esto mantiene la independencia tecnológica y hace el filtrado testeable.

5.2 Unidad de trabajo

Agrupar operaciones en una transacción lógica. Expone commit() y rollback() , y funciona como gestor de contexto. En el adaptador de memoria simula atomicidad mediante copia y confirmación; en Postgres delega en la transacción real.

Importa por una razón concreta: una derivación crea una transición, publica un evento y encola una notificación. Los tres deben ocurrir o ninguno.

5.3 Notificador

send(destinatario, plantilla, contexto) -> ResultadoEnvio

El puerto no sabe si el canal es WhatsApp, SMS o correo. El destinatario declara su canal preferido; el adaptador resuelve el transporte.

La implementación simulada almacena cada envío con su contenido renderizado y lo expone en un endpoint de inspección. **Durante la demostración esto es superior a un envío real:** el jurado ve el mensaje exacto que recibiría la familia, sin depender de la red del auditorio.

5.4 Bus de eventos

```
publish(evento) -> None
subscribe(tipo_evento, manejador) -> None
```

Implementación en proceso para el prototipo; cola distribuida en producción. El código que publica no distingue entre ambas.

5.5 Reloj

```
now() -> datetime
```

Puerto de una sola operación y de valor desproporcionado. Permite que los tests controlen el tiempo sin manipular el sistema, y permite que la demostración **avance el reloj varias semanas para disparar las alertas en vivo**. Sin este puerto, esa demostración es imposible.

El adaptador simulado añade `advance(delta)` y `set(instante)`, expuestos únicamente en modo demostración.

5.6 Almacenamiento de archivos

```
store(contenido, metadatos) -> ReferenciaArchivo
retrieve(referencia) -> bytes
signed_url(referencia, expiracion) -> str
```

Necesario para los videos breves que las familias graban en casa y para documentos adjuntos. La implementación simulada guarda en memoria y devuelve URLs ficticias.

6. Adaptadores

6.1 Matriz de implementaciones

Puerto	Adaptador de prototipo	Adaptador de producción
Repositorios	Diccionarios en memoria	PostgreSQL
Unidad de trabajo	Copia y confirmación	Transacción de base de datos
Notificador	Registro inspeccionable	WhatsApp Business API / SMS
Bus de eventos	Despacho en proceso	Cola de mensajes
Reloj	Simulado y controlable	Reloj del sistema
Almacenamiento	Memoria + URL ficticia	Object storage

6.2 Selección por configuración

```
NEURO_REPOSITORY=memory|postgres
NEURO_NOTIFIER=recording|whatsapp|sms
NEURO_EVENT_BUS=inprocess|queue
NEURO_CLOCK=simulated|system
NEURO_FILE_STORAGE=memory|object
NEURO_MODE=demo|production
```

`config.py` valida estas variables con Pydantic Settings al arrancar. Una combinación inválida —por ejemplo, reloj simulado en modo producción— aborta el arranque con un mensaje explícito. Es preferible fallar al iniciar que descubrir el problema en operación.

6.3 Composition root

`container.py` es el **único** archivo autorizado a importar adaptadores concretos. Lee la configuración, construye las instancias y las registra. Todo el resto del código recibe abstracciones.

Esta restricción se verifica con `import-linter`: si algún módulo fuera del composition root importa de `adapters/`, el build falla.

7. Servicios de aplicación

Cada servicio es un caso de uso. Recibe puertos por constructor, orquesta el dominio y publica eventos. **No contiene reglas de negocio** —esas viven en el dominio— y **no conoce HTTP**.

Servicio	Responsabilidad
ScreeningService	Aplicar cuestionario, persistir resultado, crear caso si procede
ReferralService	Generar derivación, transicionar el caso, notificar destino
SchedulingService	Programar, reprogramar y confirmar citas; agrupar sesiones de evaluación
CasefileService	Consolidar la vista del caso según el rol solicitante
NotificationService	Traducir eventos en mensajes con plantilla
AlertEngine	Evaluar reglas temporales y emitir alertas
MetricsService	Calcular tiempos entre estados y agregados de gestión

7.1 Agrupación de sesiones de evaluación

Merece mención propia porque atiende un problema concreto del desafío: la evaluación neuropsicológica requiere entre cuatro y siete sesiones, y para una familia de provincia eso significa entre cuatro y siete viajes.

El servicio recibe las sesiones requeridas, la disponibilidad de los profesionales y el perfil de desplazamiento de la familia, y propone bloques que concentren sesiones en la menor cantidad de días posible. Es un problema de asignación acotado, no un optimizador general: para el prototipo, una heurística que priorice días compartidos es suficiente y demostrable.

7.2 Motor de alertas

- Se ejecuta de tres formas, con **la misma lógica** en las tres:
- Tarea periódica en segundo plano durante la ejecución normal
 - Endpoint manual `POST /admin/alerts/run` para disparo controlado en la demostración
 - Invocación directa desde los tests

Que las tres compartan código no es un detalle: garantiza que lo que se demuestra es exactamente lo que corre en operación.

7.3 Métricas de trazabilidad

- Derivadas del historial de transiciones. Este es el argumento de gestión del proyecto: ninguna institución dispone hoy de estos tiempos sobre sus propios casos.
- Tiempo de detección a derivación
 - Tiempo de derivación a asignación de cita
 - Tiempo de cita a inicio de evaluación
 - Duración de la evaluación completa
 - Tiempo total de detección a inicio de terapia
 - Tasa de inasistencia y distribución de motivos
 - Tasa de abandono por etapa
 - Distribución geográfica de casos y de tiempos

Cada métrica se calcula desde el timeline, sin tablas paralelas que puedan desincronizarse.

8. Capa de API

8.1 Convenciones

Aspecto	Decisión
Estilo	REST sobre recursos, con acciones explícitas donde el verbo HTTP no alcanza
Versionado	Prefijo de ruta <code>/api/v1</code>
Formato	JSON exclusivamente
Fechas	ISO 8601 en UTC
Identificadores	UUID v7 (ordenables temporalmente)
Paginación	Basada en cursor, no en desplazamiento
Errores	RFC 7807 (<code>application/problem+json</code>)
Idempotencia	Cabecera <code>Idempotency-Key</code> en operaciones de escritura
Correlación	Cabecera <code>X-Request-Id</code> propagada a logs y eventos

La paginación por cursor y la idempotencia no son refinamientos opcionales: el sistema opera con conectividad intermitente, donde los reintentos son la norma y no la excepción.

8.2 Superficie de endpoints

Personal de salud del primer nivel

```
GET    /api/v1/screening/catalog?age_months={n}
POST   /api/v1/screening/applications
GET    /api/v1/screening/applications/{id}
POST   /api/v1/referrals
GET    /api/v1/health-worker/cases
GET    /api/v1/health-worker/cases/{id}
POST   /api/v1/health-worker/cases/{id}/notes
```

Familia

```
GET    /api/v1/family/cases/{id}/journey
GET    /api/v1/family/cases/{id}/next-step
GET    /api/v1/family/appointments
POST   /api/v1/family/appointments/{id}/confirm
POST   /api/v1/family/appointments/{id}/decline
GET    /api/v1/family/care-plan/{case_id}
POST   /api/v1/family/care-plan/{case_id}/activities/{activity_id}/complete
POST   /api/v1/family/uploads
GET    /api/v1/family/guidance?topic={t}
```

El endpoint de declinación de cita **exige un motivo tipado** (económico, distancia, horario laboral, salud, otro). Ese dato alimenta directamente el Insight 5 del desafío: la inasistencia no equivale a falta de compromiso, y sin capturar el motivo esa afirmación es indemostrable.

Equipo especializado

```
GET    /api/v1/specialist/cases
GET    /api/v1/specialist/cases/{id}
POST   /api/v1/specialist/cases/{id}/transitions
POST   /api/v1/specialist/cases/{id}/notes
POST   /api/v1/specialist/cases/{id}/diagnosis
POST   /api/v1/specialist/cases/{id}/care-plan
POST   /api/v1/specialist/cases/{id}/counter-referral
GET    /api/v1/specialist/alerts
POST   /api/v1/specialist/appointments
POST   /api/v1/specialist/appointments/plan-evaluation-block
GET    /api/v1/specialist/metrics
```

Administración y demostración

```
POST   /api/v1/admin/alerts/run
POST   /api/v1/admin/clock/advance
POST   /api/v1/admin/clock/reset
GET    /api/v1/admin/notifications
POST   /api/v1/admin/seed/reset
```

Este grupo se registra únicamente cuando `NEURO_MODE=demo` . En producción los endpoints no existen —no están protegidos, están ausentes.

Sistema

```
GET    /health
GET    /health/ready
GET    /api/v1/system/info
```

8.3 Contrato de errores

Toda respuesta de error sigue RFC 7807:

```
{
  "type": "https://neuroalianza.pe/errors/invalid-transition",
  "title": "Transición de estado no permitida",
  "status": 409,
  "detail": "Un caso en estado DETECTADO no puede pasar a EN_TERAPIA",
  "instance": "/api/v1/specialist/cases/018f.../transitions",
  "request_id": "01J8...",
  "errors": []
}
```

Los tipos de error forman un catálogo cerrado y documentado. Las excepciones de dominio se traducen a este formato en un único manejador registrado en la aplicación; **ningún endpoint construye respuestas de error por su cuenta**.

8.4 Modelos de entrada y salida

Los esquemas Pydantic son distintos de las entidades de dominio, siempre. Esto evita que un cambio interno rompa el contrato público y que un campo interno se filtre en una respuesta.

Reglas obligatorias:

- Configuración `extra="forbid"` en todos los modelos de entrada: un campo desconocido es un error, no algo que se ignora en silencio
- Restricciones expresadas en el tipo (rangos, longitudes, patrones), no en validaciones dispersas
- Enumeraciones para todo valor de conjunto cerrado; nunca cadenas libres
- Ejemplos declarados en cada esquema — alimentan la documentación OpenAPI y las pruebas de contrato
- Cada respuesta tiene un modelo declarado explícitamente en el decorador de ruta

8.5 Autorización por rol

Cuatro roles: `HEALTH_WORKER` , `FAMILY` , `SPECIALIST` , `ADMIN` .

El control no se limita al endpoint. **La proyección del caso depende del rol**: la familia recibe una vista en lenguaje sencillo, sin notas clínicas crudas; el especialista recibe el expediente completo; el personal de origen recibe estado y contrarreferencia de los casos que él derivó.

Esta proyección diferenciada vive en `CasefileService` , no en la capa HTTP, porque es una regla de negocio y debe ser testeable sin cliente HTTP.

9. Contrato de API y pruebas de contrato

9.1 El esquema como fuente de verdad

FastAPI genera el esquema OpenAPI a partir de las anotaciones. Ese esquema es el contrato entre backend y frontend, y se trata como un artefacto versionado:

1. El esquema se exporta a `contracts/openapi.json` mediante un comando del proyecto
2. El archivo se versiona en el repositorio
3. CI regenera el esquema y **falla si difiere del versionado**

La consecuencia práctica: nadie modifica el contrato sin que el cambio aparezca explícitamente en la revisión de código. Con tres personas trabajando en paralelo, esto elimina la clase entera de fallos por desajuste entre frontend y backend.

9.2 Schemathesis

Schemathesis toma el esquema OpenAPI y genera automáticamente casos de prueba que lo estresan: valores límite, tipos incorrectos, campos ausentes, cadenas extremas, caracteres especiales. Verifica que las respuestas reales cumplan lo que el esquema promete.

Verificaciones activadas:

Verificación	Qué detecta
<code>not_a_server_error</code>	Ausencia de respuestas 5xx ante entradas inválidas
<code>status_code_conformance</code>	Códigos devueltos declarados en el esquema
<code>content_type_conformance</code>	Tipo de contenido coincidente
<code>response_schema_conformance</code>	Cuerpo conforme al modelo declarado
<code>response_headers_conformance</code>	Cabeceras requeridas presentes
<code>negative_data_rejection</code>	Entradas inválidas rechazadas con 4xx, no aceptadas
<code>ignored_auth</code>	Endpoints protegidos que responden sin credenciales

Pruebas con estado. Schemathesis puede encadenar llamadas siguiendo los enlaces declarados en OpenAPI: crear un tamizaje, derivar el caso resultante, asignar una cita. Esto ejercita la máquina de estados a través de la API real y descubre transiciones que deberían rechazarse y no se rechazan.

Integración:

- Ejecución contra la aplicación ASGI en memoria, sin levantar servidor
- Se ejecuta en CI en cada cambio
- Cada fallo encontrado se convierte en un test de regresión permanente
- Se fija una semilla para reproducibilidad y se registran los contraejemplos

Presupuesto de ejecución. El conjunto completo se ejecuta en CI. Localmente se usa una configuración reducida en número de ejemplos, para que el ciclo de desarrollo no se vuelva lento.

9.3 Otros niveles de contrato

- **Frontend contra esquema.** El cliente del frontend se genera desde `openapi.json` . Nadie escribe llamadas a mano.
- **Adaptadores contra puertos.** Cada puerto tiene una **suite de conformidad** compartida: un conjunto de tests que toda implementación debe pasar. El adaptador de memoria y el de Postgres ejecutan exactamente los mismos tests. Esto garantiza que sustituir uno por otro no cambia el comportamiento observable —que es la promesa central de la arquitectura.

10. Estrategia de pruebas

10.1 Distribución

Nivel	Proporción	Alcance	Velocidad
Unitarias de dominio	~60 %	Máquina de estados, tamizaje, reglas de alerta	milisegundos
Servicios	~20 %	Casos de uso con adaptadores en memoria	milisegundos
Conformidad de puertos	~5 %	Cada adaptador contra su contrato	variable
API	~10 %	Endpoints con cliente ASGI	rápido
Contrato (Schemathesis)	~5 %	Generado desde el esquema	moderado

El dominio concentra la mayoría de los tests porque concentra la mayoría de las reglas. Si el grueso de los tests requiere levantar HTTP, la lógica está en la capa equivocada.

10.2 Pruebas basadas en propiedades

Para el núcleo, además de casos concretos, se verifican propiedades con Hypothesis:

- Ninguna secuencia de transiciones válidas alcanza un estado inválido
- El timeline siempre queda ordenado cronológicamente
- El motor de tamizaje nunca devuelve un resultado sin nivel de riesgo asignado
- Una edad fuera de todo rango del catálogo produce un error explícito, nunca un resultado silencioso
- Reglas de alerta idempotentes: ejecutar el motor dos veces sobre el mismo estado no duplica alertas

Las pruebas basadas en propiedades encuentran los casos límite que nadie pensó en escribir. En una máquina de estados, esa es exactamente la clase de fallo que importa.

10.3 Umbrales de cobertura

Ámbito	Mínimo
domain/	95 %
services/	90 %
ports/	100 %
adapters/memory/	90 %
api/	85 %
Global	90 %

Cobertura de ramas habilitada. El build falla por debajo del umbral. La cobertura no mide calidad, pero un descenso brusco señala código que entró sin prueba.

10.4 Convenciones

- Un archivo de test por módulo, con estructura espejo
- Nombres que describen el comportamiento esperado, no el método invocado
- Datos de prueba contruidos con factorías, nunca literales dispersos
- Prohibido `sleep`: el control del tiempo pasa siempre por el puerto de reloj
- Sin red en las pruebas — el bloqueo se configura globalmente
- Cada test es independiente y puede ejecutarse en cualquier orden

11. Calidad de código

11.1 Ruff en modo estricto

Configuración base: **todas las reglas activadas**, con exclusiones justificadas caso por caso.

```
[tool.ruff]
target-version = "py312"
line-length = 100
src = ["app", "tests"]

[tool.ruff.lint]
select = ["ALL"]
ignore = [
    "D203",      # incompatible con D211
    "D213",      # incompatible con D212
    "COM812",    # conflicto con el formateador
    "ISC001",    # conflicto con el formateador
]

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["S101", "PLR2004", "ANN201", "D103"]
"app/api/routes/*" = ["B008"]      # Depends() en valores por defecto
"app/seed/*" = ["E501"]

[tool.ruff.lint.pydocstyle]
convention = "google"

[tool.ruff.lint.mccabe]
max-complexity = 8

[tool.ruff.lint.flake8-type-checking]
strict = true

[tool.ruff.lint.isort]
known-first-party = ["app"]
required-imports = ["from __future__ import annotations"]

[tool.ruff.lint.flake8-bugbear]
extend-immutable-calls = ["fastapi.Depends", "fastapi.Query", "fastapi.Path"]

[tool.ruff.format]
quote-style = "double"
docstring-code-format = true
```

Familias de reglas de mayor impacto en este proyecto:

Prefijo	Familia	Por qué importa aquí
ANN	Anotaciones de tipo	Base para que MyPy estricto sea efectivo
S	Seguridad (bandit)	Datos de salud de menores de edad
B	Bugbear	Errores sutiles frecuentes
RUF	Reglas específicas de Ruff	Detecta patrones peligrosos con mutabilidad
PT	Estilo de pytest	Consistencia de la suite
C90	Complejidad ciclomática	Límite 8: fuerza a partir la lógica del dominio
TCH	Bloques de solo-tipado	Mantiene limpias las dependencias en runtime
ERA	Código comentado	Prohibido
ARG	Argumentos no usados	Detecta firmas desactualizadas
SLF	Acceso a privados	Protege las fronteras entre módulos
TID	Imports acoplados	Refuerza las fronteras de capa

Regla del proyecto: una exclusión nueva requiere un comentario que explique el motivo. Silenciar sin justificar es una observación de revisión.

11.2 MyPy en modo estricto

```
[tool.mypy]
python_version = "3.12"
strict = true

# Más allá de strict
disallow_any_unimported = true
disallow_any_expr = false           # impracticable con Pydantic
disallow_any_decorated = false      # impracticable con decoradores de FastAPI
disallow_any_explicit = true
disallow_any_generics = true
disallow_subclassing_any = true
warn_unreachable = true
warn_no_return = true
strict_equality = true
strict_bytes = true
extra_checks = true
enable_error_code = [
    "redundant-expr",
    "possibly-undefined",
    "truthy-bool",
    "truthy-iterable",
    "ignore-without-code",
    "unused-awaitable",
    "explicit-override",
    "mutable-override",
    "unimported-reveal",
]
show_error_codes = true
pretty = true

[[tool.mypy.overrides]]
module = "tests.*"
disallow_untyped_defs = true

[[tool.mypy.overrides]]
module = ["app.domain.*"]
disallow_any_explicit = true
warn_return_any = true
```

Aclaración sobre dos opciones desactivadas. `disallow_any_expr` y `disallow_any_decorated` son incompatibles en la práctica con Pydantic y con los decoradores de FastAPI: activarlas produce miles de errores en código correcto. Se documentan aquí para dejar constancia de que la decisión fue deliberada y no un olvido.

Prácticas de tipado exigidas:

- Tipos de identidad diferenciados (`NewType`) para cada clase de identificador. Un `CasoId` y un `PacienteId` no deben ser intercambiables: el sistema maneja media docena de identificadores y confundirlos es un fallo silencioso.
- `Literal` y `Enum` para todo conjunto cerrado de valores
- `Final` para constantes
- Modelos de dominio inmutables (`frozen=True`)
- `Protocol` para los puertos, no herencia
- `assert_never` en la exhaustividad de la máquina de estados: agregar un estado sin manejarlo se convierte en un error de tipos, no en un fallo en producción
- Prohibido `# type: ignore` sin código específico (`ignore-without-code` lo garantiza)

11.3 Verificación de fronteras entre módulos

`import-linter` convierte la regla de dependencia en una prueba automática:

```
[[tool.importlinter.contracts]]
name = "Arquitectura en capas"
type = "layers"
layers = ["app.api", "app.services", "app.ports", "app.domain"]

[[tool.importlinter.contracts]]
name = "El dominio no depende de nada"
type = "forbidden"
source_modules = ["app.domain"]
forbidden_modules = ["app.api", "app.services", "app.adapters", "app.ports",
                    "fastapi", "pydantic", "sqlalchemy", "httpx"]

[[tool.importlinter.contracts]]
name = "Adaptadores solo desde el composition root"
type = "forbidden"
source_modules = ["app.api", "app.services", "app.domain"]
forbidden_modules = ["app.adapters"]

[[tool.importlinter.contracts]]
name = "Módulos de dominio independientes entre sí"
type = "independence"
modules = ["app.domain.screening", "app.domain.casefile",
          "app.domain.scheduling", "app.domain.alerts"]
```

El último contrato es el más exigente y el más valioso: obliga a que los contextos del dominio se comuniquen por eventos y no por importación directa. Es la diferencia entre modularidad declarada y modularidad verificada.

11.4 Verificaciones adicionales

Herramienta	Función	Bloquea el build
ruff format	Formato	Sí
ruff check	Linting	Sí
mypy	Tipos	Sí
import-linter	Fronteras de capa	Sí
pytest	Pruebas	Sí
coverage	Umbrales	Sí
schemathesis	Contrato de API	Sí
bandit	Seguridad	Sí
pip-audit	Vulnerabilidades en dependencias	Advertencia
interrogate	Cobertura de docstrings	Advertencia

Todo se ejecuta en pre-commit localmente y se repite en CI. Lo que pasa en local pasa en CI: no hay dos conjuntos de reglas.

11.5 Comandos del proyecto

```
make install      # instalar dependencias y hooks
make run          # arrancar con adaptadores en memoria
make lint         # ruff check + format --check
make types        # mypy
make arch         # import-linter
make test         # pytest con cobertura
make contract     # exportar openapi.json + schemathesis
make check        # todo lo anterior
make seed         # recargar datos de demostración
```

make check es la puerta de entrada. Si pasa localmente, pasa en CI.

12. Configuración y operación

12.1 Configuración tipada

Todas las variables se declaran en un modelo de Pydantic Settings con prefijo `NEURO_`. Ninguna lectura directa de variables de entorno fuera de ese modelo.

Parámetros por familia:

- **Selección de adaptadores** — los seis descritos en 6.2
- **Umbrales de alerta** — días de derivación estancada, semanas sin actividad, horas de anticipación de recordatorio
- **Tamizaje** — versión activa del catálogo
- **Agenda** — duración por tipo de sesión, ventana máxima de agrupación
- **Seguridad** — secretos, expiración de tokens, orígenes permitidos
- **Observabilidad** — nivel de log, formato

12.2 Registro y observabilidad

Logs estructurados en JSON, con `request_id` propagado desde la cabecera de entrada hasta los eventos publicados.

Nunca se registran: nombres de pacientes, documentos de identidad, contenido de notas clínicas, contactos. Los logs referencian identificadores, no personas. Esta es una restricción absoluta cuando el sistema maneja datos de salud de menores de edad.

12.3 Datos de demostración

El dataset precargado debe contar una historia completa y verosímil, no ser ruido aleatorio:

- Casos en cada estado de la máquina, incluyendo abandono y reactivación
- Al menos un caso con recorrido íntegro, de detección a alta
- Casos de distintas regiones, con tiempos de traslado contrastantes
- Casos que ya disparan alertas al arrancar
- Distribución de motivos de inasistencia que permita mostrar la métrica
- Un caso "estrella" para narrar en el pitch, con nombre, región e historia coherente

Un dataset bien construido es la mitad de una buena demostración.

13. Seguridad y protección de datos

Ámbito	Medida
Minimización	Solo se almacenan los datos necesarios para la ruta de atención
Autenticación	Tokens de vida corta, refresco separado
Autorización	Verificación por rol y por pertenencia al caso
Auditoría	Todo acceso a un expediente queda registrado
Transporte	TLS obligatorio en producción
Reposo	Cifrado de campos identificatorios
Registros	Sin datos personales, en ninguna circunstancia
Consentimiento	Registro explícito del consentimiento del cuidador para tratamiento y almacenamiento
Retención	Política de retención y borrado declarada por tipo de dato

Aun cuando el prototipo no maneje datos reales, estas decisiones deben estar **presentes en el diseño y documentadas**. Ante un jurado hospitalario, la ausencia de una postura sobre protección de datos de menores es un déficit visible.

14. Secuencia de implementación

Fase	Entregable	Criterio de término
1	Esqueleto, configuración, puertos, adaptadores en memoria	<code>make check</code> pasa sobre un proyecto vacío
2	Dominio: estados y tamizaje, con pruebas	Cobertura $\geq$ 95 % en <code>domain/</code>
3	Servicios y bus de eventos	Flujo detección $\rightarrow$ derivación probado extremo a extremo
4	API: rutas de los tres roles	<code>openapi.json</code> exportado y estable
5	Motor de alertas y reloj simulado	Alertas disparables por endpoint
6	Métricas y datos de demostración	Panel de gestión con datos coherentes
7	Schemathesis y endurecimiento	Suite de contrato en verde
8	Adaptador Postgres	Suite de conformidad de puertos en verde con ambos adaptadores

Las fases 1 a 4 son el prototipo demostrable. Las fases 5 a 7 son las que diferencian la propuesta. La fase 8 solo aplica si sobra tiempo, y por diseño no requiere tocar nada de lo anterior.

15. Resumen de decisiones

Decisión	Fundamento
Puertos y adaptadores	Arrancar sin infraestructura; migrar sin reescribir
Adaptadores en memoria por defecto	Demostración sin red ni credenciales
Puerto de reloj	Simular semanas de operación frente al jurado
Notificador con registro inspeccionable	Mostrar el mensaje sin depender de la red del auditorio
Timeline inmutable	Trazabilidad y métricas desde una única fuente
Comunicación por eventos	Agregar comportamiento sin tocar el flujo principal
Motor de tamizaje puro y versionado	Auditabilidad ante criterio clínico
Esquema OpenAPI versionado	Frontend y backend en paralelo sin desajustes
Schemathesis en CI	El contrato se verifica solo, no por inspección
Ruff con ALL y MyPy estricto	Los errores aparecen al escribir, no en la demostración
<code>import-linter</code>	La modularidad es verificada, no declarada
Suites de conformidad de puertos	Sustituir un adaptador no cambia el comportamiento